

Step 6 — What's Not Touched in the Basic Flow (and Why)

Steps 1–5 cover the complete "buy → claim → paid" trace. This step covers everything on the requested list that genuinely does **not** execute during that trace, and says precisely where each piece actually fits instead — because implying a false connection would be worse than naming the gap directly.

Part A — `Vault.sol` and `PriceOracle.sol` : prerequisite, not live-trace

Where they actually fit: *before* Step 2, not during it. Both Alice and `ClaimRegistry` needed dUSD in hand before anything in Steps 1–5 could happen — Step 2 assumed Alice already held 33 dUSD, and Step 5 assumed `ClaimRegistry` already held 221+ dUSD. Neither of those balances came from nowhere.

Concrete example — how `ClaimRegistry` got its 221+ dUSD:

1. The insurer deposits collateral: `Vault.depositCollateral(50 ether)` (50 units of `twETH`, the mock collateral token). Contract: `Vault` (a proxy address not yet in this document's address book, since it belongs to a separate deployment — the CDP module has its own dedicated Timelock, deliberately separate from `ClaimRegistry / InsurancePolicy / OracleAdapter`'s governance). Traversal: `IERC20(collateralToken).safeTransferFrom(insurer, Vault, 50 ether)`, increments `positions[insurer].collateralAmount` from 0 to 50 ether.
2. The insurer mints against it: `Vault.mintStableCoin(50_000 ether)`. Internally, `Vault` calls `PriceOracle.getPrice()` — say it returns 2_000 ether (i.e. "\$2,000 per unit of collateral"). `Vault` computes `collateralValueUsd = 50 * 2_000 = 100_000 ether`, checks the resulting ratio against `minCollateralRatioBps` (e.g. 150%: $100_000 / 50_000 = 200\%$, healthy), and — this is the **only** call anywhere in this whole document set to `StableCoin`'s privileged `mint` function — calls `StableCoin.mint(insurer, 50_000 ether)`.
3. The insurer transfers 221+ ether of that newly-minted dUSD to `ClaimRegistry`'s address (`0x3333...cccc`) — a plain `StableCoin.transfer` call, identical in kind to every other transfer in Steps 2 and 5, just funding the contract that will later pay Alice.

Alice's own 33 dUSD (spent in Step 2) came from the structurally identical flow — deposit collateral, mint, keep the dUSD instead of transferring it onward. **This is a real, currently-unaddressed gap in the actual frontend:** there's no guided "get dUSD" flow anywhere in the UI; a policyholder would need to interact with `Vault` directly today.

Part B — `DICSGovernanceToken.sol`, `DICSGovernor.sol`, and EIP-2612: a separate domain entirely

Not touched, and structurally cannot be touched, by anything in Steps 1–5. These two contracts govern *only* the CDP module's risk parameters —

`Vault.minCollateralRatioBps`, `Vault.liquidationPenaltyBps`, `PriceOracle`'s admin role — via their own dedicated `TimelockController`, completely separate from the Timelock that governs `ClaimRegistry / InsurancePolicy / OracleAdapter`. This separation is deliberate: "who can change CDP collateral ratios" and "who can approve insurance claims" are different trust domains with no reason to share a single point of control.

EIP-2612 appears only inside `DICSGovernanceToken.sol` — it's the gasless-approval standard `ERC20Permit` implements, which `ERC20Votes` depends on for its own nonce-tracking machinery (voting-power delegation can be authorized via a signature instead of a separate on-chain transaction). It has no relationship whatsoever to Alice's premium payment or

claim — those use plain EIP-20 `approve / transferFrom` (Step 2), not EIP-2612.

Concrete example of when this domain *would* activate: if the CDP module's collateralization ratio needed raising from 150% to 180% (a real, plausible risk-management action), a `DICSGovernanceToken` holder would call `DICSGovernor.propose(...)` targeting `Vault.updateRiskParams(18_000, ...)`, other token holders would `castVote`, and — only after both the voting period and the CDP Timelock's own delay elapse — `DICSGovernor.execute(...)` would actually change the parameter. None of this has any code path into `ClaimRegistry` at all.

Part C — `ClaimGasPaymaster.sol` and EIP-4337: an optional variant, not the base trace

In Steps 1–5, Alice paid her own transaction gas as a plain externally-owned account (EOA) — the normal way. **Neither this contract nor EIP-4337 executed anywhere in that trace.**

Where it would fit, concretely, if used: had Alice's wallet been a smart-contract account and the app chosen to sponsor her gas, Step 3 (`submitClaim`) would instead be wrapped like this, *before* the actual call:

1. Alice's smart account packages the intended `submitClaim` call into a `PackedUserOperation` and sends it to a bundler — not directly to the chain.
2. The bundler calls the canonical `EntryPoint` contract, which calls `ClaimGasPaymaster._validatePaymasterUserOp(userOp, userOpHash, maxCost)` — checks Alice's rolling daily gas-sponsorship total (`sponsoredToday[0x1111...aaaa][dayBucket]`) against `dailySponsorshipCapWei`, reverting if it would be exceeded.
3. `EntryPoint` executes the `UserOperation` — this is what actually triggers `ClaimRegistry.submitClaim`, **identical in every respect to Step 3 above** once it runs; the paymaster only changes who pays for gas, never what the transaction does.
4. `EntryPoint` calls `ClaimGasPaymaster._postOp(mode, context, actualGasCost, actualUserOpFeePerGas)` — records the real gas cost against Alice's daily total.

Worth repeating from this project's own contract documentation: this paymaster does not verify the sponsored call is actually `submitClaim` specifically — as built, it would sponsor *any* operation from a given sender up to their daily cap. Restricting it further is real, unfinished work, not a detail this document is skipping over.

Summary table — every requested item, one line each

Item	Role in Alice's 33 – to–221 trace
EIP-20	Every dUSD movement: Steps 2, 5, and Part A's funding flow
EIP-1967	Storage-slot standard underlying every proxy call in Steps 2–5
EIP-1822	The UUPS upgrade pattern those same proxies implement
EIP-712	Step 4 — signing and verifying the oracle's attestation
EIP-191	Step 1 only — Alice's off-chain wallet login, nothing on-chain
EIP-2612	Not used — <code>DICSGovernanceToken</code> only, a separate governance domain
EIP-4337	Not used in the base trace — optional gas-sponsorship variant only
<code>ClaimRegistry.sol</code>	Steps 3, 4, 5 — claim lifecycle and payout
<code>InsurancePolicy.sol</code>	Steps 2, 3 — subscription, premium, policy checks
<code>OracleAdapter.sol</code>	Step 4 — signature verification and forwarding

OracleAdapter.sol	Step 4 — signature verification and forwarding
StableCoin.sol	Steps 2, 5 (transfers); Part A step 2 (the only <code>mint</code> call in this whole set)
Vault.sol	Part A only — funding, not the live claim flow
PriceOracle.sol	Part A step 2 only — read once, by <code>Vault</code> , during funding
DICSGovernanceToken.sol	Not used — Part B only
DICSGovernor.sol	Not used — Part B only
ClaimGasPaymaster.sol	Not used in the base trace — Part C variant only